



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

AIRLINE RESERVATION MANAGEMENT SYSTEM USING C

1. PSEUDOCODE

START

1. Initialize passenger, flight, and booking arrays.
2. Set the initial number of passengers, flights, and bookings to zero.
3. Display the main menu.
4. Repeat until the user selects Exit:
 1. Register Passenger
 2. Add Flight
 3. View Passenger Details
 4. View Flight Details
 5. Book Ticket
 6. View Booking Details
 7. Find Passenger
 8. Find Flight
 9. Modify Passenger
 10. Cancel Ticket
 11. Sort Bookings
 12. Check Available Seats
 13. Store Records
 14. Exit
5. Read the user's choice.
6. Register passenger details and generate a passenger ID.
7. Add flight information and initialize seat availability.
8. Display passenger and flight records when requested.
9. For ticket booking, verify the passenger and flight, check seat availability, assign a seat, calculate fare, and create a booking ID.
10. Search passenger or flight records using their IDs.
11. Update passenger information when required.
12. Cancel a booking and release its seat.
13. Sort booking records using Bubble Sort.
14. Save records using file handling.

15. Display an error for an invalid selection.

STOP

2. IMPLEMENTATION & RESULTS

2.1 Implementation

The Airline Reservation Management System was developed in C as a menu-driven application. It manages passenger registration, flight details, seat allocation, ticket booking, cancellation and record storage. The program is divided into functions so that each operation can be developed and tested independently.

C Concept	Application
Structures	Store passenger, flight and booking information
Arrays	Maintain multiple records
Functions	Perform individual system operations
Pointers	Access and modify stored records
Loops	Repeat menu and record-processing operations
Conditional Statements	Validate reservations and availability
Switch Case	Control the main menu
Searching	Locate passenger, flight and booking records
Sorting	Arrange booking records
File Handling	Save and retrieve records

2.2 Data Structures

The system uses separate structures for Passenger, Flight and Booking. Passenger stores personal details such as ID, name, age, gender, phone number and email. Flight stores flight number, source, destination, travel date, departure time, fare and seat information. Booking stores booking ID, passenger ID, flight number, seat number, ticket amount and booking status.

2.3 Flight and Seat Management

Seat information is maintained for every flight. An available seat is represented by [], while an occupied seat is represented by [X]. During booking, the program searches for an available seat and marks it as occupied. When a ticket is cancelled, the assigned seat becomes available again.

Example Seat Status:

```
1 [X] 2 [ ] 3 [X] 4 [ ]
5 [ ] 6 [X] 7 [ ] 8 [ ]
9 [ ] 10 [ ] 11 [X] 12 [ ]
```

2.4 Functions Used

registerPassenger() – creates passenger records.

addFlight() – stores flight details.

displayPassengers() – displays registered passengers.

displayFlights() – displays flight information.

bookTicket() – performs ticket reservation and seat allocation.
displayBookings() – shows booking records.
searchPassenger() – finds a passenger using the passenger ID.
searchFlight() – finds a flight using the flight number.
updatePassenger() – modifies passenger information.
cancelTicket() – cancels a reservation and releases the seat.
sortBookings() – arranges booking records.
saveRecords() – stores records in a file.

2.5 Searching and Sorting

Linear Search is used to locate passenger, flight and booking records. The program compares the required ID with records one by one until a match is found. Bubble Sort is used to arrange booking records in ascending or descending order according to the selected field.

Example:

Before Sorting: B105 ₹6500, B101 ₹3500, B108 ₹5000, B103 ₹2500

After Sorting: B103 ₹2500, B101 ₹3500, B108 ₹5000, B105 ₹6500

2.6 File Handling

File handling is used to retain passenger, flight and booking information after the program is closed. The functions fopen(), fwrite(), fread() and fclose() can be used to write and retrieve binary records. This provides persistent storage for the application.

2.7 Results

The system was tested for passenger registration, flight creation, seat allocation, ticket booking, passenger searching, flight searching, passenger modification, ticket cancellation, sorting, seat checking, saving and loading records. The tested operations produced the expected results.

Sample Booking Confirmation

Booking ID : B1001

Passenger ID : P101

Flight No. : AI205

From : Chennai

To : Delhi

Seat No. : 12

Ticket Amount : Rs. 5200

Status : CONFIRMED

Sample Seat Output

1. Flight creation

```
=====
                        ADD NEW FLIGHT
=====

Flight Number : AI202
Source       : Chennai
Destination  : Delhi
Date (DD MM YYYY): 15 09 2026
Departure Time: 09:30
Base Fare    : 5500

-----

Flight created successfully!
Flight ID    : F001
Total Seats  : 50
Available    : 50
-----
```

2. Actual aircraft seat map

```
=====
                        AIRCRAFT SEAT MAP
=====

Flight : AI202
Route  : Chennai -> Delhi

Legend: [ ] Available  [X] Booked

          WINDOW      AISLE      WINDOW

          1  2  3  4  5  6  7  8  9 10
          -----
ROW 1 | [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ]
ROW 2 | [ ] [ ] [ ] [X] [ ] [ ] [ ] [ ] [ ] [ ]
ROW 3 | [ ] [ ] [X] [ ] [ ] [ ] [ ] [ ] [ ] [ ]
ROW 4 | [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ]
ROW 5 | [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ]

-----

Available Seats : 47
Booked Seats    : 3
Total Seats     : 50
-----
```

3. Booking confirmation

```
=====
                        BOOKING CONFIRMED
=====
Reservation ID : R0004
Passenger      : Anand
Flight         : AI202
Route          : Chennai -> Delhi
Travel Date    : 15-09-2026
Departure      : 09:30
Seat           : Row 2 - Seat 4
Priority        : BUSINESS
Payment        : UPI
Ticket Amount  : Rs. 6352.50
Status         : Confirmed
=====
```

4. Cancellation

```
=====
                        RESERVATION CANCELLED
=====
Reservation : R0004
Released Seat: Row 2 - Seat 4
Status      : Cancelled
=====
```

5. File Storage Result

```
=====
DATA SAVED SUCCESSFULLY
File: airline_data.dat
Passengers   : 2
Flights      : 1
Reservations : 3
=====
```

6.File Handling Output

```
=====
                        DATA SAVED SUCCESSFULLY
=====

File: airline_data.dat
Passengers   : 2
Flights      : 1
Reservations : 2
=====
```

3. TEST CASES

Test Case	Operation	Expected Result	Status
TC01	Register Passenger	Passenger record created	PASS
TC02	Add Flight	Flight information stored	PASS
TC03	View Passenger	Passenger details displayed	PASS
TC04	View Flight	Flight details displayed	PASS
TC05	Book Ticket	Seat assigned and booking confirmed	PASS
TC06	Search Passenger	Correct passenger displayed	PASS
TC07	Search Flight	Correct flight displayed	PASS
TC08	Update Passenger	Passenger details modified	PASS
TC09	Cancel Ticket	Booking cancelled and seat released	PASS
TC10	Check Seats	Available seats displayed	PASS
TC11	Sort Bookings	Records arranged correctly	PASS
TC12	Save Records	Data stored successfully	PASS
TC13	Load Records	Previous data retrieved	PASS
TC14	Invalid Input	Error message displayed	PASS

4. DATA STRUCTURE / ALGORITHM COMPARISON

Alternative 1 – Array of Structures + Linear Search + Bubble Sort

The selected implementation uses arrays of structures to store passenger, flight and booking information. Linear Search is used for locating records and Bubble Sort is used for arranging booking data.

Advantages:

- Simple to understand and implement.
- Suitable for a small academic application.
- Easy to demonstrate in C.
- Requires limited programming complexity.
- Direct access to records using indexes.

Disadvantages:

- Array capacity is limited.
- Searching becomes slower as records increase.
- Bubble Sort requires more comparisons for large datasets.
- Not ideal for large commercial systems.

Alternative 2 – Linked List + Binary Search Tree

A more advanced design could use linked lists for dynamic storage and a binary search tree for faster record lookup.

Advantages:

- Dynamic memory allocation.
- Flexible record growth.
- Better suited for larger datasets.
- Faster searching can be achieved with balanced tree structures.

Disadvantages:

- More pointer manipulation is required.
- Implementation is more complex.
- Debugging and memory management are harder.

Final Design Selection

The Array of Structures with Linear Search and Bubble Sort approach was selected because the project is intended for a limited academic dataset. It provides a clear demonstration of fundamental C programming concepts while keeping the program easy to understand and test.

5. COMPLEXITY ANALYSIS

Operation	Method	Time Complexity
Register Passenger	Array insertion	O(1)
Add Flight	Array insertion	O(1)
Search Passenger	Linear Search	O(n)
Search Flight	Linear Search	O(n)

Search Booking	Linear Search	O(n)
Update Passenger	Linear Search	O(n)
Cancel Ticket	Linear Search	O(n)
Check Seat	Array traversal	O(n)
Sort Bookings	Bubble Sort	O(n ²)
Save Records	File Write	O(n)
Load Records	File Read	O(n)

6. INDIVIDUAL CONTRIBUTION OF C

I contributed to the development of the Airline Reservation Management System using C by studying the requirements and helping organize the application into passenger, flight, booking, seat and cancellation modules.

I worked on the design of the data structures used for storing passenger, flight and booking information. I also contributed to passenger registration, passenger display, passenger searching and passenger information modification.

Another part of my contribution involved flight and seat management. I worked on storing flight information and maintaining the availability status of seats. During ticket booking, the system verifies the passenger and flight, assigns an available seat and stores the booking details.

I contributed to the ticket cancellation process by locating the booking, changing its status and releasing the occupied seat. I also worked with searching and sorting operations using Linear Search and Bubble Sort.

Functions were used to divide the application into smaller modules, making the program easier to understand and maintain. I also contributed to file handling using `fopen()`, `fwrite()`, `fread()` and `fclose()` to store and retrieve records.

Through this project, I gained practical knowledge of structures, arrays, functions, pointers, loops, conditional statements, searching, sorting and file handling in C. The project helped me understand how fundamental programming concepts can be combined to create a practical reservation system.

7. CONCLUSION

The Airline Reservation Management System Using C was successfully developed as a menu-driven application for performing basic airline reservation activities. The system supports passenger registration, flight management, seat allocation, ticket booking, searching, updating and cancellation.

The project demonstrates the practical use of structures, arrays, functions, pointers, loops, conditional statements, searching, sorting and file handling. These concepts provide a strong foundation for developing larger information-management applications.

The seat management feature helps track occupied and available seats, while the cancellation facility makes previously booked seats available again. File handling provides persistent storage so that records can be retained between program executions.

Overall, the project provides a simple and effective implementation of an airline reservation workflow in C. Future improvements could include database connectivity, user authentication, online payment support, graphical interfaces and advanced search facilities.